Universidade Federal de Viçosa
Campus Florestal

CCF 131 - Fundamentos da Teoria da Computação

Trabalho Prático 3

Aline Cristina Santos Silva 5791
Marcus Eduardo Resende Melo - 5779
Luana Tavares Anselmo - 5364
Gustavo Luca Ribeiro da Silva 5787
Gabriel Henrique Bramante Guimarães 5794
Iago Guimarães Falcão - 5795

Professor: Daniel Mendes Barbosa

Sumário

1 Introdução **2**

2 Organização e Compilação do Projeto **3**

2.1 Linguagem escolhida 3

2.2 Organização do Projeto 3

2.3 Compilação e Execução 3

2.4 Dependências 4

3 Desenvolvimento **5**

3.1 Ingredientes Utilizados 5

3.2 AFD 7

3.3 APD 9

3.4 Mealy - Extra 11

3.5 Moore - Extra 13

3.6 Turing - Extra 15

3.7 Interface - Extra 16

4 Resultados **17**

4.1 Via Terminal 17

4.2 Via Interface 19

5 Conclusão **24**

6 Referências **25**

1. Introdução

A construção de autômatos finitos e outros modelos computacionais é um dos pilares fundamentais da Teoria da Computação. Compreender os limites e capacidades de diferentes tipos de máquinas nos permite reconhecer, analisar e resolver problemas formais com precisão e eficiência. Este trabalho prático foi proposto com o objetivo de aplicar esses conceitos em um contexto lúdico e criativo: a produção automatizada de poções mágicas.

Inspirados pelo universo fantástico da feitiçaria, nosso grupo foi encarregado de auxiliar uma tradicional família de bruxos e bruxas no controle de qualidade das suas misturas, que se tornaram populares e altamente demandadas. Para isso, implementamos uma interface interativa que simula a produção de poções utilizando diferentes tipos de autômatos. Por meio da adição controlada de ingredientes, a interface permite verificar se uma determinada receita é válida ou se ocorreram erros na mistura — sempre respeitando as regras definidas em cada máquina.

O sistema proposto contempla a implementação obrigatória de Autômatos Finitos Determinísticos (AFD) e de Autômatos com Pilha (APD), além de extensões opcionais utilizando Máquina de Turing, Máquina de Moore e Máquina de Mealy, que enriquecem ainda mais a proposta ao explorar diferentes níveis de poder computacional.

Cada máquina foi projetada para refletir as capacidades e limitações de seu modelo, tornando visível ao usuário como diferentes tipos de autômatos reagem à adição de ingredientes, verificam propriedades, manipulam estruturas de memória e retornam um resultado. A escolha dos ingredientes, das regras e das reações foi feita de forma a proporcionar uma experiência divertida e educativa, ao mesmo tempo em que evidencia a diferença entre linguagens regulares, livres de contexto e computações mais complexas.

Além da lógica computacional, também foi desenvolvida uma interface gráfica temática, que simula um verdadeiro Laboratório de Poções Mágicas. Nela, o usuário pode interagir com cada tipo de máquina, visualizar mensagens e reações da mistura e até acompanhar animações ou imagens mágicas que ilustram o processo de produção. A interface foi construída com cuidado para ser intuitiva, envolvente e condizente com o universo do projeto.

Com isso, este trabalho não apenas atende aos requisitos acadêmicos da disciplina de Fundamentos da Teoria da Computação, como também propõe uma abordagem prática, criativa e interdisciplinar do conteúdo, conectando teoria, programação e design em uma aplicação funcional e envolvente.

2. Organização e Compilação do Projeto

2.1. Linguagem escolhida

A linguagem de programação utilizada neste trabalho foi o **Python**. A escolha foi unânime entre os integrantes do grupo, principalmente por conta de sua flexibilidade, legibilidade e ampla comunidade de suporte. Além disso, o Python oferece bibliotecas poderosas para o desenvolvimento de interfaces gráficas, como o `Tkinter`, que foi utilizada neste projeto.

Optamos por criar uma interface gráfica interativa para tornar a experiência do usuário mais envolvente, ilustrando visualmente a construção das poções. Entretanto, também implementamos versões em terminal dos autômatos, permitindo testes diretos e independentes da interface. O projeto, portanto, suporta os dois modos de execução: via terminal e via interface gráfica.

2.2. Organização do Projeto

O projeto foi estruturado em pastas para facilitar a organização, leitura e manutenção do código:

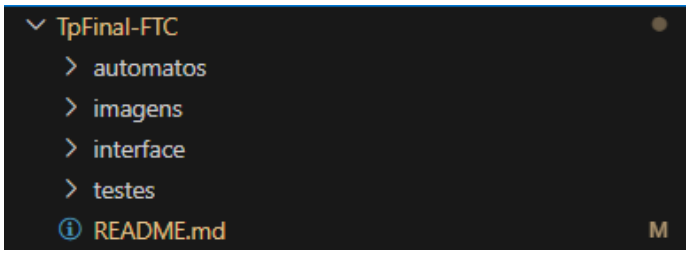

Figura 1. Automatos: Contém as implementações dos autômatos em Python (AFD, APD, Moore, Mealy e Turing). Entradas: Subpasta dentro de automatos/ que armazena os arquivos .txt de entrada para os autômatos AFD e APD. Interface: Contém o código da interface gráfica desenvolvida com Tkinter. Testes: Scripts de testes para execução dos autômatos via terminal. Cada autômato possui um script próprio. Imagens: Contém imagens utilizadas na interface. README.md: Arquivo contendo instruções de uso, dependências e comandos para execução do projeto.

2.3. Compilação e Execução

Para executar o projeto, os seguintes comandos podem ser utilizados a partir da raiz do diretório:

- **Executar a interface gráfica:**
  `python -m interface.app`

- **Executar os testes no terminal:**
  - AFD: `python -m testes.testAfd`
  - APD: `python -m testes.testApd`
  - Moore: `python -m testes.testMoore`
  - Mealy: `python -m testes.testMealy`
  - Turing: `python -m testes.testTuring`

```
## TpFinal-FTC
    Executar Interface: python -m interface.app
    Executar os testes(Afd): python -m testes.testAfd
    Executar os testes(Apd): python -m testes.testApd
    Executar os testes(Moore): python -m testes.testMoore
    Executar os testes(Mealy): python -m testes.testMealy
    Executar os testes(Turing): python -m testes.testTuring


#### Instalar a seguinte biblioteca:
pip install graphviz
```

Figura 2. Arquivo README com os comandos para execução do programa.

2.4. Dependências

Para o funcionamento completo do projeto, é necessário instalar a seguinte dependência:

`pip install graphviz`

Essa biblioteca é usada para visualizar as transições e diagramas de estados.

3. Desenvolvimento

O desenvolvimento do projeto foi dividido em etapas bem definidas, a fim de garantir uma construção modular, escalável e que explorasse diferentes tipos de autômatos, conforme proposto no trabalho.

Inicialmente, foram implementados os dois autômatos obrigatórios: o **Autômato Finito Determinístico (AFD)** e o **Autômato com Pilha (APD)**. Ambos foram desenvolvidos com base nos requisitos da especificação, garantindo a leitura de pelo menos três ingredientes e a presença de transições completas, incluindo estados de erro.

No entanto, buscando ir além dos requisitos mínimos e evidenciar o domínio dos conceitos abordados ao longo da disciplina, o grupo decidiu expandir o projeto com a implementação de funcionalidades extras, que incluem:

- Uma **Máquina de Mealy**, responsável por simular a produção de poções com respostas visuais a cada ingrediente adicionado;
- Uma **Máquina de Moore**, utilizada para montar um Frankenstein mágico, com saídas associadas aos estados;
- Uma **Máquina de Turing**, que verifica se uma palavra é um palíndromo, demonstrando o maior poder computacional entre as máquinas utilizadas;
- E uma **interface gráfica interativa**, desenvolvida com `tkinter`, que permite a execução de todas as máquinas de forma visual, amigável e intuitiva.

Esses elementos extras foram pensados para complementar o trabalho, oferecendo uma visão mais completa sobre os diferentes tipos de autômatos, suas aplicações práticas e suas capacidades computacionais distintas. Cada uma dessas implementações será detalhada nas seções a seguir.

3.1. Ingredientes Utilizados

Foram definidos 6 ingredientes principais, cada um representado por um símbolo textual na entrada das máquinas:

Ingrediente	Símbolo
Água	`agua`
Pétala	`petala`
Óleo	`oleo`
Pó de fada	`po`
Raiz mágica	`raiz`
Cristal	`cristal`

Tabela 1. Esses ingredientes foram usados nas diversas combinações válidas e inválidas para simular a criação de poções, explorando regras diferentes em cada tipo de autômato.

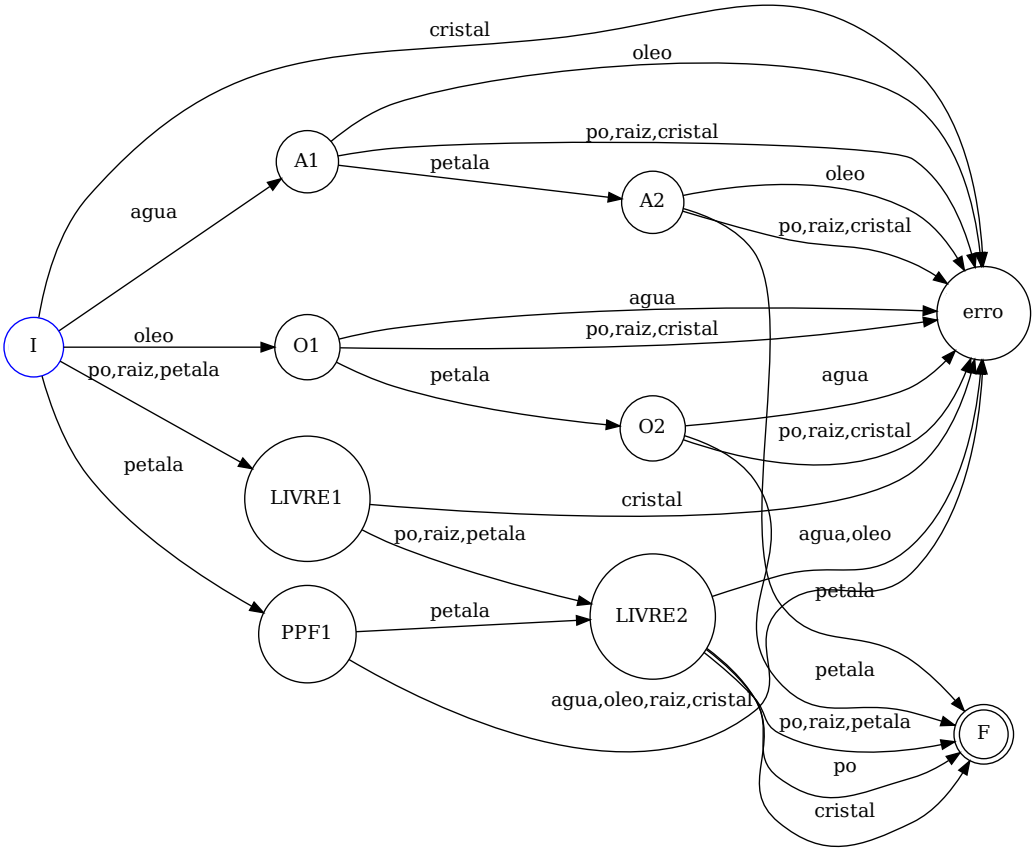

Figura 3. Regras dos ingredientes escolhidos representadas por um AFD visual.

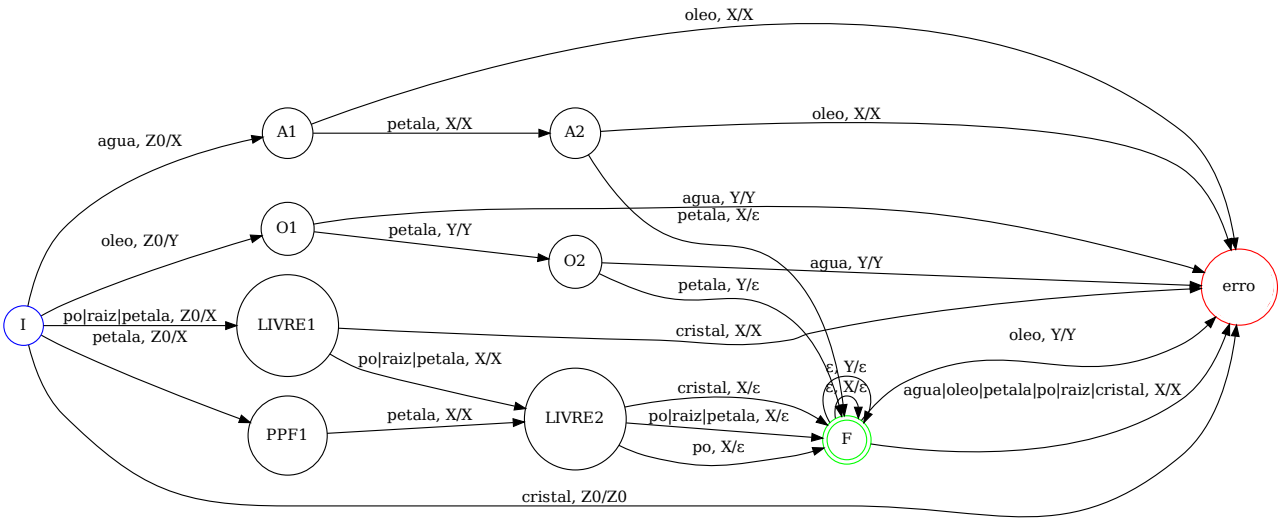

Figura 4. Regras dos ingredientes escolhidos representadas por um APD visual.

3.2. AFD

A classe AFD foi responsável por simular o comportamento de um Autômato Finito Determinístico, processando ingredientes de uma poção de acordo com uma sequência válida prédefinida. A máquina é alimentada por um arquivo .txt contendo sua descrição formal, com estados, transições e alfabeto.

```python
class AFD:
    def __init__(self, caminho_arquivo):
        self.estados = []
        self.estado_inicial = ""
        self.estado_final = ""
        self.transicoes = {}  # Ex: {('I', 'a'): 'ing1'}
        self.estado_atual = ""
        self.estado_erro = "erro"
        self.carregar_arquivo(caminho_arquivo)
```

Figura 5. Essa função é chamada na criação do objeto e realiza a leitura e carregamento do arquivo de entrada (IngredientesAfd.txt). Inicializa os atributos principais da máquina, como o estado atual, estados finais e tabela de transições.

```python
def carregar_arquivo(self, caminho_arquivo):
    with open(caminho_arquivo, 'r') as arquivo:
        linhas = [linha.strip() for linha in arquivo if linha.strip()]

    for i, linha in enumerate(linhas):
        if linha.startswith("Q:"):
            self.estados = linha[3:].split()
        elif linha.startswith("I:"):
            self.estado_inicial = linha[3:].strip()
            self.estado_atual = self.estado_inicial
        elif linha.startswith("F:"):
            self.estado_final = linha[3:].strip()
        elif linha == "---":
            break
        else:
            partes = linha.split("->")
            origem = partes[0].strip()
            destino, simbolos_str = partes[1].split("|")
            destino = destino.strip()
            simbolos = simbolos_str.strip().split()
            for simbolo in simbolos:
                self.transicoes[(origem, simbolo)] = destino
```

Figura 6. Essa função lê o arquivo .txt linha por linha e interpreta os dados para configurar a máquina. As transições são armazenadas como um dicionário Python, onde a chave é uma tupla (estado_atual, simbolo) e o valor é o estado_destino.

```python
def resetar(self):
    self.estado_atual = self.estado_inicial
```

Figura 7. Responsável por reinicializar a máquina ao estado inicial, permitindo múltiplas execuções sem recriar o objeto.

```python
def transitar(self, simbolo):
    chave = (self.estado_atual, simbolo)
    if chave in self.transicoes:
        proximo_estado = self.transicoes[chave]
        print(f"> {self.estado_atual} --[{simbolo}]--> {proximo_estado}")
        self.estado_atual = proximo_estado
    else:
        print(f"> Transição inválida com '{simbolo}'. Indo para o estado de erro.")
        self.estado_atual = self.estado_erro
```

Figura 8. Dada uma entrada (ingrediente), essa função realiza a transição para o próximo estado com base no estado atual e na tabela de transições.Se a transição for válida, o estado é atualizado.Caso contrário, a máquina entra no estado de erro (erro).Também imprime no terminal a transição executada, o que facilita o rastreio da execução durante testes.

```python
def executar(self):
    print("🧪 Início da produção de poção!")
    while True:
        simbolo = input("Insira o símbolo do próximo ingrediente: ").strip()
        self.transitar(simbolo)

        if self.estado_atual == self.estado_erro:
            break

        continuar = input("Deseja inserir mais um ingrediente? (s/n): ").strip().lower()
        if continuar != 's':
            break

    # Verificação final
    if self.estado_atual == self.estado_final:
        print("\n✅ Poção criada com sucesso!")
    else:
        print("\n❌ Erro na mistura.")
```

Figura 9. Essa função executa a simulação no terminal, perguntando ao usuário os ingredientes que deseja inserir.A cada inserção, é feita uma transição.O usuário é questionado se deseja continuar.Ao final, verifica se o estado atual é igual ao estado final.Se sim, a poção foi criada com sucesso. Caso contrário, houve erro na mistura.

3.3. APD

A classe `APD` foi implementada com o objetivo de representar um autômato de pilha não determinístico de forma determinística e funcional, sendo capaz de processar sequências de ingredientes e simular reações empilhadas na pilha mágica. A leitura da máquina é realizada a partir de um arquivo de texto formatado, semelhante ao AFD, contendo os estados, transições, alfabeto e símbolos de pilha.

```python
class APD:
    def __init__(self, caminho_arquivo):
        self.estados = set()
        self.estado_inicial = None
        self.estado_atual = None
        self.estado_final = None
        self.simbolos_pilha = set()
        self.alfabeto = set()
        self.transicoes = {}  # chave: (estado, simbolo_entrada,
        self.pilha = []
        self._carregar_arquivo(caminho_arquivo)
```

Figura 10. É o construtor da classe. Ele é chamado quando um novo APD é criado. Inicializa os atributos do autômato (estados, pilha, transições etc.) e chama a função que carrega os dados do autômato a partir de um arquivo .txt.

```python
def _carregar_arquivo(self, caminho):
    with open(caminho, 'r', encoding='utf-8') as arquivo:
        linhas = [linha.strip() for linha in arquivo if linha.strip() and not linha.startswith('#')]

    secao_transicoes = False
    for linha in linhas:
        if linha.startswith('Q:'):
            self.estados = set(linha[2:].strip().split())
        elif linha.startswith('I:'):
            self.estado_inicial = linha[2:].strip()
            self.estado_atual = self.estado_inicial
        elif linha.startswith('F:'):
            self.estado_final = linha[2:].strip()
        elif linha.startswith('P:'):
            self.simbolos_pilha = set(linha[2:].strip().split())
        elif linha.startswith('Alfabeto:'):
            self.alfabeto = set(linha[len('Alfabeto:'):].strip().split())
        elif linha == '---':
            break
        elif '->' in linha:
            esquerda, direita = linha.split('->')
            esquerda = esquerda.strip()
            direita = direita.strip()
            estado, simbolo, topo = [x.strip() for x in esquerda.split(',')]
            novo_estado, novo_topo = [x.strip() for x in direita.split(',')]
            self.transicoes[(estado, simbolo, topo)] = (novo_estado, novo_topo)

    # Símbolo inicial da pilha
    self.pilha.append('Z0')

    # print("Transições carregadas:")
    #for k, v in self.transicoes.items():
        #print(f"{k} -> {v}")
```

Figura 11. Carrega toda a definição do APD a partir de um arquivo externo. Lê as informações do arquivo (estados, símbolos, transições, etc.) e armazena tudo em estruturas internas (conjuntos, listas e dicionário). Também define o símbolo base da pilha.

```python
def transicionar(self, simbolo):
    topo = self.pilha[-1] if self.pilha else 'Z0'
    chave = (self.estado_atual, simbolo, topo)

    if chave not in self.transicoes:
        print(f"Transição inválida para ({self.estado_atual}, {simbolo}, {topo})")
        return False

    novo_estado, nova_pilha = self.transicoes[chave]
    print(f"Transição: ({self.estado_atual}, {simbolo}, {topo}) -> ({novo_estado}, {nova_pilha})")
    self.estado_atual = novo_estado

    if nova_pilha == 'ε':
        if self.pilha:
            self.pilha.pop()
    else:
        self.pilha.append(nova_pilha)

    return True
```

Figura 12. Realiza uma única transição do autômato com base no símbolo lido. Verifica se há uma transição válida considerando o estado atual, o símbolo lido e o topo da pilha. Se existir: Atualiza o estado atual. Empilha ou desempilha de acordo com a nova configuração. Retorna True indicando sucesso. Se não existir, retorna False.

```python
def executar(self):
    print("\nIniciando simulação do APD...\n")
    while True:
        simbolo = input("Insira o próximo ingrediente (ou 'fim' para encerrar): ").strip()
        if simbolo == 'fim':
            # Processar transições epsilon automaticamente até o fim possível
            while True:
                topo = self.pilha[-1] if self.pilha else 'Z0'
                chave_epsilon = (self.estado_atual, 'ε', topo)
                if chave_epsilon in self.transicoes:
                    novo_estado, nova_pilha = self.transicoes[chave_epsilon]
                    print(f"Transição epsilon automática: ({self.estado_atual}, ε, {topo}) -> ({novo_estado}, {nova_pilha})")
                    self.estado_atual = novo_estado
                    if nova_pilha == 'ε':
                        self.pilha.pop()
                    else:
                        self.pilha.append(nova_pilha)
                else:
                    break
            break

        if simbolo not in self.alfabeto:
            print("Ingrediente inválido!")
            continue
        if not self.transicionar(simbolo):
            print("Mistura falhou. Reações instáveis.")
            return

    if self.estado_atual == self.estado_final and (self.pilha[-1] == 'Z0' or len(self.pilha) == 0):
        print("\n✅ Poção criada com sucesso!")
    else:
        print("\n❌ Poção incompleta ou instável.")
```

Figura 13. Inicia a execução interativa do APD com o usuário (modo terminal). Pede os símbolos (ingredientes) um por um. Quando o usuário digita fim, a função tenta aplicar automaticamente transições vazias restantes. Por fim, verifica se o estado atual é o final e se a pilha está vazia (ou só com Z0) para aceitar ou rejeitar a poção.

3.4. Mealy - Extra

Como funcionalidade extra do trabalho, o grupo implementou uma Máquina de Mealy com o objetivo de simular visualmente e interativamente a mistura de ingredientes mágicos em uma poção. Esse autômato é capaz de responder dinamicamente a cada ingrediente inserido, alterando o estado da poção e exibindo mensagens mágicas conforme as reações provocadas.

```python
class MealyPocao:
    def __init__(self):
        self.estado = 'normal'  # estados possíveis: normal, congelada, fervendo
        self.transicoes = {
            ('normal', 'gelo'): ('congelada', '\nA poção congelou! 🧊'),
            ('normal', 'fogo'): ('fervendo', '\nA poção está fervendo! 🔥'),
            ('normal', 'agua'): ('normal', '\nÁgua adicionada, poção continua normal. 💧'),
            ('normal', 'po_magico'): ('normal', '\nPó mágico espalhado, poção brilha! ✨'),
            ('normal', 'ervas'): ('normal', '\nErvas adicionadas, aroma agradável! 🌿'),

            ('congelada', 'fogo'): ('normal', '\nA poção descongelou! 🔥❄️'),
            ('congelada', 'gelo'): ('congelada', '\nMais gelo adicionado, continua congelada. 🧊'),
            ('congelada', 'agua'): ('congelada', '\nÁgua fria adicionada, poção ainda congelada. ❄️💧'),
            ('congelada', 'po_magico'): ('congelada', '\nPó mágico não teve efeito no gelo. ❄️✨'),
            ('congelada', 'ervas'): ('congelada', '\nErvas congeladas dentro da poção. ❄️🌿'),

            ('fervendo', 'gelo'): ('normal', '\nA poção esfria e para de ferver. 🧊🔥'),
            ('fervendo', 'fogo'): ('fervendo', '\nFogo intensificado, poção mais quente! 🔥🔥'),
            ('fervendo', 'agua'): ('fervendo', '\nÁgua quente adicionada, mantém fervura. 🔥💧'),
            ('fervendo', 'po_magico'): ('fervendo', '\nPó mágico potencializa a fervura! ✨🔥'),
            ('fervendo', 'ervas'): ('fervendo', '\nErvas vaporizadas pela fervura! 🌿🔥'),
        }
        self.ingredientes = ['gelo', 'fogo', 'agua', 'po_magico', 'ervas']
```

Figura 14. Inicializa o objeto da máquina de estados, configurando o estado inicial da poção como normal. Define o dicionário transicoes que representa as transições de estados da máquina. Cada chave é uma tupla (estado_atual, ingrediente), e o valor é outra tupla (novo_estado, saída), onde: novo_estado é o estado para o qual a poção vai após adicionar o ingrediente. Saída é a mensagem que indica o resultado da ação, simulando o comportamento de uma máquina de Mealy (onde a saída depende do estado e da entrada).Lista os ingredientes válidos que podem ser processados.

```python
def processar_ingrediente(self, ingrediente):
    chave = (self.estado, ingrediente)
    if chave in self.transicoes:
        novo_estado, saida = self.transicoes[chave]
        self.estado = novo_estado
        return saida
    else:
        return "Ingrediente desconhecido ou ação inválida."

def processar(self, ingrediente):
        chave = (self.estado, ingrediente)
        if chave in self.transicoes:
            novo_estado, resposta = self.transicoes[chave]
            self.estado = novo_estado
            return resposta
        else:
            return "❌ Ingrediente não teve efeito nesse estado."
```

Figura 15. Recebe um ingrediente como entrada. Verifica se existe uma transição definida para o estado atual da poção e o ingrediente recebido. Caso exista, atualiza o estado da poção para o novo estado e retorna a mensagem correspondente. Caso contrário, retorna uma mensagem indicando que a ação ou ingrediente é inválido naquele estado. Essa função é responsável por executar a lógica principal da máquina, simulando as transições e produzindo as saídas.

```python
def iniciar_interacao(self):
    print("Bem-vindo ao Misturador de Poções Mealy!")
    print("Estado inicial da poção:", self.estado)
    print("Ingredientes disponíveis:")
    for ingr in self.ingredientes:
        print(f"- {ingr}")

    while True:
        ingrediente = input("\nDigite o ingrediente a adicionar (ou 'sair' para encerrar): ").strip().lower()
        if ingrediente == 'sair':
            print("Finalizando mistura. Estado final da poção:", self.estado)
            break
        elif ingrediente not in self.ingredientes:
            print("Ingrediente inválido. Por favor, escolha um da lista.")
            continue
        resposta = self.processar_ingrediente(ingrediente)
        print(resposta)
        print(f"Estado atual da poção: {self.estado}")
```

Figura 16. Função que inicia uma interação direta com o usuário via terminal. Exibe o estado inicial da poção e os ingredientes disponíveis para serem adicionados. Entra em um loop onde o usuário pode digitar ingredientes para modificar a poção. Para cada ingrediente digitado: Valida se o ingrediente é válido. Processa o ingrediente, atualizando o estado da poção e exibindo a mensagem de saída. Mostra o estado atual da poção. Permite encerrar a interação digitando 'sair'. Essa função facilita testes manuais e demonstra o funcionamento da máquina em tempo real.

3.5. Moore - Extra

A classe MooreFrankenstein simula uma máquina de Moore aplicada à montagem de um monstro Frankenstein. Nessa máquina, o estado atual depende exclusivamente da configuração das partes já adicionadas ao corpo, e a saída (mensagem exibida) está diretamente associada ao estado, conforme o comportamento esperado de uma máquina de Moore. O objetivo é montar completamente o corpo do monstro adicionando suas partes (cabeça, tronco, dois braços e duas pernas), acompanhando visualmente a progressão e as mudanças de estado.

```
class MooreFrankenstein:
    def __init__(self):
        self.estado = 'INICIO'
        self.partes = {
            'cabeca': 0,
            'tronco': 0,
            'braco': 0,
            'perna': 0
        }
```

Figura 17. O funcionamento da máquina começa com o método __init__, que inicializa o estado como INICIO e define um dicionário com todas as partes do corpo do monstro, cada uma com valor zero. Essas partes são cabeca, tronco, braco e perna, sendo que cabeça e tronco só podem ser adicionados uma vez, enquanto braços e pernas podem ser adicionados duas vezes.

```
def _atualizar_estado(self):
    if self.partes['cabeca'] == 1 and \
       self.partes['tronco'] == 1 and \
       self.partes['braco'] == 2 and \
       self.partes['perna'] == 2:
        self.estado = 'COMPLETO'
    elif self.partes['tronco'] == 1:
        self.estado = 'MONTANDO'
    else:
        self.estado = 'INCOMPLETO'
```

Figura 18. A lógica de atualização do estado é centralizada na função interna _atualizar_estado, que verifica, a cada adição de parte, se todas as condições para completar o monstro foram atendidas. Caso todas as partes tenham sido adicionadas corretamente (1 cabeça, 1 tronco, 2 braços e 2 pernas), o estado passa a ser COMPLETO. Se apenas o tronco foi adicionado até o momento, o estado se torna MONTANDO. Em qualquer outra configuração incompleta, o estado permanece como INCOMPLETO.

```python
def adicionar_parte(self, parte):
    if self.estado == 'COMPLETO':
        print("🧟 Frankenstein já está completo!")
        return

    if parte not in self.partes:
        print("❌ Parte inválida!")
        return

    limite = 1 if parte in ['cabeca', 'tronco'] else 2
    if self.partes[parte] < limite:
        self.partes[parte] += 1
        print(f"✅ Parte adicionada: {parte}")
    else:
        print(f"⚠️ Já há {limite} {parte}(s).")

    self._atualizar_estado()
    print(self.saida_estado())
```

Figura 19. Para adicionar uma parte, o usuário interage com o método adicionar_parte, que recebe o nome da parte como argumento. Esse método verifica se o monstro já está completo caso esteja, ele não permite mais modificações e informa que o monstro já está finalizado. Em seguida, verifica se a parte solicitada é válida e se ainda pode ser adicionada, respeitando os limites definidos. Quando uma parte é corretamente adicionada, o estado da máquina é atualizado e a saída correspondente ao novo estado é exibida automaticamente, garantindo que a máquina funcione com saídas baseadas apenas no estado atual, como define o modelo de Moore.

```python
    def saida_estado(self):
        if self.estado == 'COMPLETO':
            return self._frankenstein_visual()
        elif self.estado == 'MONTANDO':
            return f"🔧 Montagem em progresso: {self.partes}"
        else:
            return f"🧩 Ainda incompleto: {self.partes}"

    def _frankenstein_visual(self):
        # Montagem final com emojis:
        return """
🧟 Frankenstein COMPLETO!
"""
```

Figura 20. A saída gerada após cada ação é produzida pela função saida_estado, que retorna uma string com base no estado da máquina. Se o monstro estiver completo, ela chama a função _frankenstein_visual, que apresenta uma mensagem especial com emojis indicando que o monstro foi finalizado com sucesso. Nos estados intermediários, são exibidas mensagens com a quantidade de partes já montadas, reforçando visualmente o progresso da montagem.

3.6. Turing - Extra

A classe MaquinaTuring implementa uma simulação de uma máquina de Turing determinística, cuja função é verificar se uma palavra de entrada é um palíndromo,ou seja, se ela pode ser lida da mesma forma de trás para frente. Essa máquina utiliza uma fita com símbolos, um cabeçote de leitura/gravação e uma lógica de estados finitos que determinam o comportamento com base no símbolo lido e no estado atual, como definido formalmente no modelo de Turing.

```python
def executar(self):
    while True:
        simbolo = self.fita[self.posicao]

        if self.estado == 'q0':
            if simbolo not in ['X', '_']:
                self.simbolo_inicial = simbolo
                self.fita[self.posicao] = 'X'
                self.estado = 'direita'
                self.posicao += 1
            elif simbolo == 'X' or simbolo == '_':
                self.estado = 'aceita'
                break
            else:
                self.estado = 'rejeita'
                break

        elif self.estado == 'direita':
            if self.fita[self.posicao] != '_':
                self.posicao += 1
            else:
                self.posicao -= 1
                self.estado = 'verifica'

        elif self.estado == 'verifica':
            if self.fita[self.posicao] == self.simbolo_inicial:
                self.fita[self.posicao] = 'X'
                self.estado = 'volta_inicio'
                self.posicao -= 1
            elif self.fita[self.posicao] == 'X':
                self.estado = 'volta_inicio'
                self.posicao -= 1
            else:
                self.estado = 'rejeita'
                break

        elif self.estado == 'volta_inicio':
            if self.fita[self.posicao] != 'X':
                self.posicao -= 1
            else:
                self.posicao += 1
                self.estado = 'q0'
        else:
            break

    if self.estado == 'aceita':
        print("Palindromo!")
        return 'aceita'
    else:
        print("Não é um palindromo!")
        return 'rejeita'
```

Figura 21. A função principal é executar(), que implementa o comportamento da máquina. No estado inicial q0, a máquina lê o primeiro símbolo da palavra e o marca com X. Em seguida, no estado direita, ela avança até o final da fita, onde entra no estado verifica para comparar o símbolo final com o inicial marcado. Se os símbolos forem iguais, ambos são marcados e a máquina retorna ao início no estado volta_inicio, reiniciando o processo. Caso encontre um símbolo diferente, a palavra é rejeitada.

3.7. Interface - Extra

A interface gráfica do projeto foi construída com a biblioteca tkinter, e tem como objetivo centralizar e facilitar a interação do usuário com os diferentes autômatos implementados: AFD, APD, Máquina de Moore, Máquina de Mealy e Máquina de Turing. A janela principal apresenta um layout intuitivo e temático, simulando um "laboratório mágico" onde o usuário pode montar poções e criaturas a partir de autômatos formais.

```python
class App:
    def __init__(self, root):
        self.root = root
        self.root.title("✨ Laboratório de Autômatos Mágicos ✨")
        self.root.geometry("900x700")
        self.root.config(bg="#f5f3ff")

        # Título
        self.title_label = tk.Label(
            root,
            text="🧙‍♀️ Bem-vindo ao Laboratório de Poções Automatizadas!",
            font=("Garamond", 20, "bold"),
            fg="#4b0082",
            bg="#f5f3ff"
        )
        self.title_label.pack(pady=20)

        # Frame dos botões
        self.button_frame = tk.Frame(root, bg="#f5f3ff")
        self.button_frame.pack(pady=10)

        estilo_botao = {
            "font": ("Garamond", 12),
            "width": 25,
            "bg": "#dcd0ff",
            "activebackground": "#c6b4f0"
        }

        self.afd_button = tk.Button(self.button_frame, text="🧪 Montar Poção (AFD)", command=self.executar_afd, **estilo_botao)
        self.afd_button.grid(row=0, column=0, padx=5, pady=5)

        self.apd_button = tk.Button(self.button_frame, text="🧙‍♂️ Montar Poção (APD)", command=self.executar_apd, **estilo_botao)
        self.apd_button.grid(row=0, column=1, padx=5, pady=5)

        self.frank_button = tk.Button(self.button_frame, text="🧟 Montar Frankenstein (Moore)", command=self.executar_frankenstein, **estilo_botao)
        self.frank_button.grid(row=0, column=2, padx=5, pady=5)

        self.turing_button = tk.Button(self.button_frame, text="🔄 Validar Palíndromo (Turing)", command=self.executar_turing, **estilo_botao)
        self.turing_button.grid(row=1, column=0, padx=5, pady=5)

        self.mealy_button = tk.Button(self.button_frame, text="🍾 Misturar Poção (Mealy)", command=self.iniciar_mealy, **estilo_botao)
        self.mealy_button.grid(row=1, column=1, padx=5, pady=5)
        # Botão para limpar
        self.clear_button = tk.Button(self.button_frame, text="🧹 Limpar Interface", command=self.limpar_interface, **estilo_botao)
        self.clear_button.grid(row=1, column=2, padx=5, pady=5)

        # Área de saída
        self.text_output = tk.Text(root, height=25, width=100, font=("Consolas", 10), bg="#fff8fc")
        self.text_output.pack(pady=20)
        self.imagem_frankenstein = None
```

Figura 22. A função principal da interface é oferecer botões que disparam os métodos específicos de cada autômato. Cada botão corresponde a uma tarefa mágica: montar poções com AFD ou APD, criar um Frankenstein com uma máquina de Moore, misturar poções com Mealy ou validar palíndromos com a Máquina de Turing. Após a escolha do usuário, a interface solicita interações (como inserir ingredientes ou palavras) por meio de diálogos simples, exibindo os resultados passo a passo em uma área de texto abaixo, que simula um "relatório de laboratório".Além disso, a interface gerencia dinamicamente os botões de entrada para a máquina de Mealy, exibe imagens (como no caso da montagem do Frankenstein) e conta com um botão de limpeza que reinicia o ambiente, permitindo novas execuções. Assim, a interface não só organiza visualmente a execução dos autômatos, como também torna a experiência mais lúdica, didática e acessível.

4. Resultados

A seguir serão mostrados os arquivos .txt que estão sendo utilizados para testar o APD e AFD no programa. As regras criadas por eles estão disponíveis no autonômos visuais que foram apresentados acima na documentação.

```
Q: I A1 A2 O1 O2 LIVRE1 LIVRE2 LIVRE3 PPF1 CRISTAL1 F
I: I
F: F

I -> A1 | agua
A1 -> A2 | petala
A2 -> F | petala
I -> O1 | oleo
O1 -> O2 | petala
O2 -> F | petala


A1 -> erro | oleo
A2 -> erro | oleo
O1 -> erro | agua
O2 -> erro | agua


I -> LIVRE1 | po raiz petala
LIVRE1 -> LIVRE2 | po raiz petala
LIVRE2 -> F | po raiz petala


I -> PPF1 | petala
PPF1 -> LIVRE2 | petala
LIVRE2 -> F | po


I -> LIVRE1 | petala raiz po
LIVRE1 -> LIVRE2 | petala raiz po
LIVRE2 -> F | cristal


I -> erro | cristal
LIVRE1 -> erro | cristal


F_agua -> erro | agua oleo petala po raiz cristal
F_oleosa -> erro | agua oleo petala po raiz cristal
F_livre -> erro | agua oleo petala po raiz cristal
F_magica -> erro | agua oleo petala po raiz cristal


A1 -> erro | po raiz cristal
A2 -> erro | po raiz cristal
O1 -> erro | po raiz cristal
O2 -> erro | po raiz cristal
PPF1 -> erro | agua oleo raiz cristal
LIVRE2 -> erro | agua oleo
```

Figura 23. Arquivo .txt AFD

```
I: I
F: F
P: X Y Z0
Alfabeto: agua oleo petala po raiz cristal

Transições:

# Água → petala → petala
I, agua, Z0 -> A1, X
A1, petala, X -> A2, X
A2, petala, X -> F, ε

# Óleo → petala → petala
I, oleo, Z0 -> O1, Y
O1, petala, Y -> O2, Y
O2, petala, Y -> F, ε

# Bloqueia mistura agua + oleo
A1, oleo, X -> erro, X
A2, oleo, X -> erro, X
O1, agua, Y -> erro, Y
O2, agua, Y -> erro, Y

# po + raiz + petala em qualquer ordem (empilha ingredientes)
I, po, Z0 -> LIVRE1, X
I, raiz, Z0 -> LIVRE1, X
I, petala, Z0 -> LIVRE1, X

LIVRE1, po, X -> LIVRE2, X
LIVRE1, raiz, X -> LIVRE2, X
LIVRE1, petala, X -> LIVRE2, X

LIVRE2, po, X -> F, ε
LIVRE2, raiz, X -> F, ε
LIVRE2, petala, X -> F, ε

# petala + petala + po → mágica
I, petala, Z0 -> PPF1, X
PPF1, petala, X -> LIVRE2, X
LIVRE2, po, X -> F, ε

# Cristal apenas como terceiro ingrediente
LIVRE2, cristal, X -> F, ε

# Cristal em 1º ou 2º → erro
I, cristal, Z0 -> erro, Z0
LIVRE1, cristal, X -> erro, X

# Finais ignoram tudo
F, agua, X -> erro, X
F, oleo, Y -> erro, Y
F, petala, X -> erro, X
F, po, X -> erro, X
F, raiz, X -> erro, X
```

Figura 24. Arquivo .txt APD

4.1. Via Terminal

```
C:\Users\aline\OneDrive\Documentos\facul\5 semestre\ftc\TpFinal-FTC>python -m testes.testAfd
🧪 Início da produção de poção!
Insira o símbolo do próximo ingrediente: agua
> I --[agua]--> A1
Deseja inserir mais um ingrediente? (s/n): s
Insira o símbolo do próximo ingrediente: petala
> A1 --[petala]--> A2
Deseja inserir mais um ingrediente? (s/n): s
Insira o símbolo do próximo ingrediente: petala
> A2 --[petala]--> F
Deseja inserir mais um ingrediente? (s/n): n

✅ Poção criada com sucesso!
```

Figura 25. Saída AFD via terminal com a poção sendo criada com sucesso.

```
C:\Users\aline\OneDrive\Documentos\facul\5 semestre\ftc\TpFinal-FTC>python -m testes.testApd

Iniciando simulação do APD...

Insira o próximo ingrediente (ou 'fim' para encerrar): raiz
Transição: (I, raiz, Z0) -> (LIVRE1, X)
Insira o próximo ingrediente (ou 'fim' para encerrar): po
Transição: (LIVRE1, po, X) -> (LIVRE2, X)
Insira o próximo ingrediente (ou 'fim' para encerrar):
Ingrediente inválido!
Insira o próximo ingrediente (ou 'fim' para encerrar): petala
Transição: (LIVRE2, petala, X) -> (F, ε)
Insira o próximo ingrediente (ou 'fim' para encerrar): fim
Transição epsilon automática: (F, ε, X) -> (F, ε)

✅ Poção criada com sucesso!
```

Figura 26. Saída APD via terminal com a poção sendo criada com sucesso.

```
C:\Users\aline\OneDrive\Documentos\facul\5 semestre\ftc\TpFinal-FTC>python -m testes.testMoore
Digite a parte a adicionar (cabeca, tronco, braco, perna) ou 'sair': cabeca
✅ Parte adicionada: cabeca
🧩 Ainda incompleto: {'cabeca': 1, 'tronco': 0, 'braco': 0, 'perna': 0}
Digite a parte a adicionar (cabeca, tronco, braco, perna) ou 'sair': tronco
✅ Parte adicionada: tronco
🔧 Montagem em progresso: {'cabeca': 1, 'tronco': 1, 'braco': 0, 'perna': 0}
Digite a parte a adicionar (cabeca, tronco, braco, perna) ou 'sair': perna
✅ Parte adicionada: perna
🔧 Montagem em progresso: {'cabeca': 1, 'tronco': 1, 'braco': 0, 'perna': 1}
Digite a parte a adicionar (cabeca, tronco, braco, perna) ou 'sair': braco
✅ Parte adicionada: braco
🔧 Montagem em progresso: {'cabeca': 1, 'tronco': 1, 'braco': 1, 'perna': 1}
Digite a parte a adicionar (cabeca, tronco, braco, perna) ou 'sair': perna
✅ Parte adicionada: perna
🔧 Montagem em progresso: {'cabeca': 1, 'tronco': 1, 'braco': 1, 'perna': 2}
Digite a parte a adicionar (cabeca, tronco, braco, perna) ou 'sair': braco
✅ Parte adicionada: braco

🧟 Frankenstein COMPLETO!
```

Figura 27. Saída Máquina de Moore com o Frankstein sendo criado com sucesso.

```
C:\Users\aline\OneDrive\Documentos\facul\5 semestre\ftc\TpFinal-FTC>python -m testes.testMealy
Bem-vindo ao Misturador de Poções Mealy!
Estado inicial da poção: normal
Ingredientes disponíveis:
- gelo
- fogo
- agua
- po_magico
- ervas

Digite o ingrediente a adicionar (ou 'sair' para encerrar): agua

Água adicionada, poção continua normal. 💧
Estado atual da poção: normal

Digite o ingrediente a adicionar (ou 'sair' para encerrar): fogo

A poção está fervendo! 🔥
Estado atual da poção: fervendo

Digite o ingrediente a adicionar (ou 'sair' para encerrar): gelo

A poção esfria e para de ferver. 🧊🔥
Estado atual da poção: normal

Digite o ingrediente a adicionar (ou 'sair' para encerrar): ervas

Ervas adicionadas, aroma agradável! 🌿
Estado atual da poção: normal
```

Figura 28. Saída Máquina de Meealy com as transições a partir dos ingredientes sendo criado com sucesso.

```
C:\Users\aline\OneDrive\Documentos\facul\5 semestre\ftc\TpFinal-FTC>python -m testes.testTuring
Digite uma palavra para verificar se é um palíndromo (Sem X): 1221
Palindromo!
```

Figura 29. Saída Máquina de Turing mostrando que a palavra escrita é um palíndromo.

4.2. Via Interface

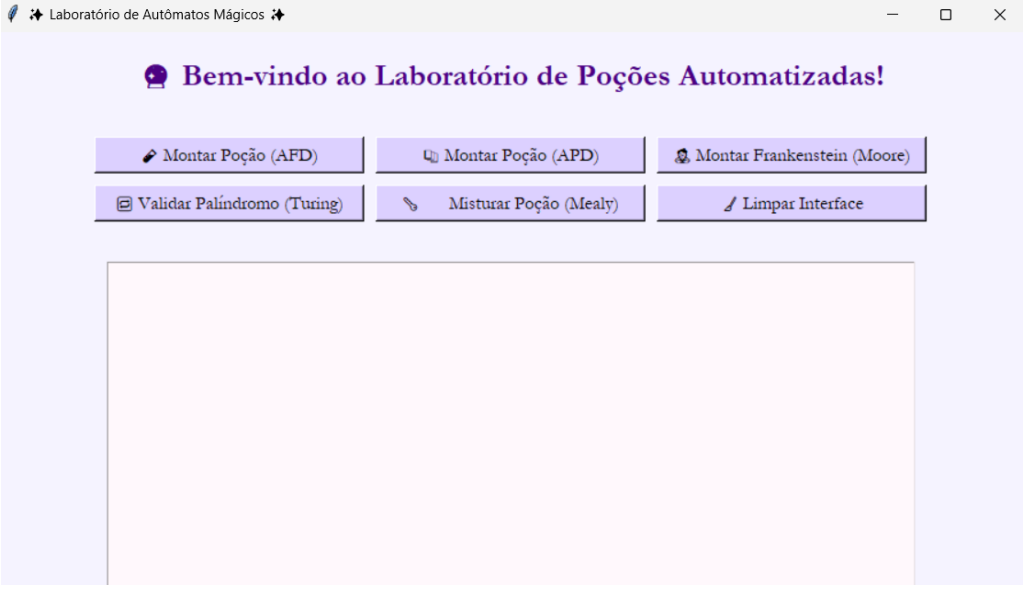

Figura 30. Tela Inicial da Interface.

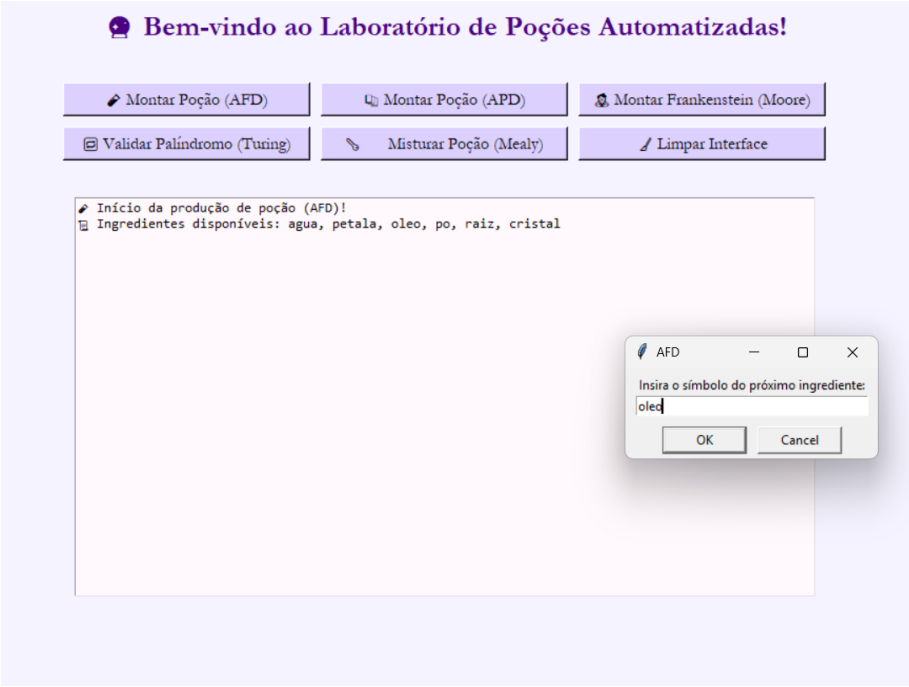

Figura 31. Adicionando ingredientes no AFD.

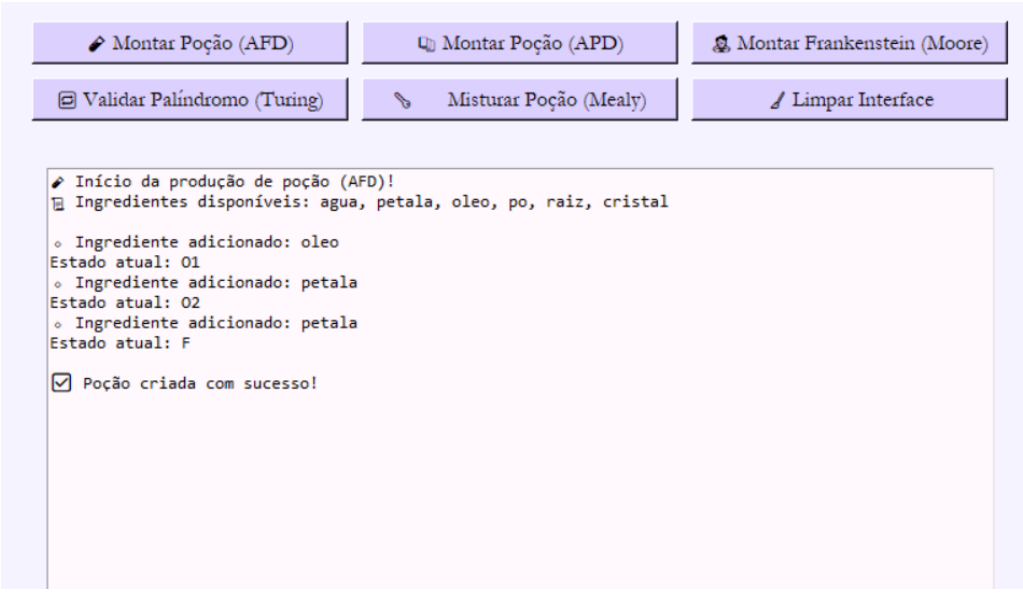

Figura 32. Depois de adicionar todos os ingredientes a poção é criada.

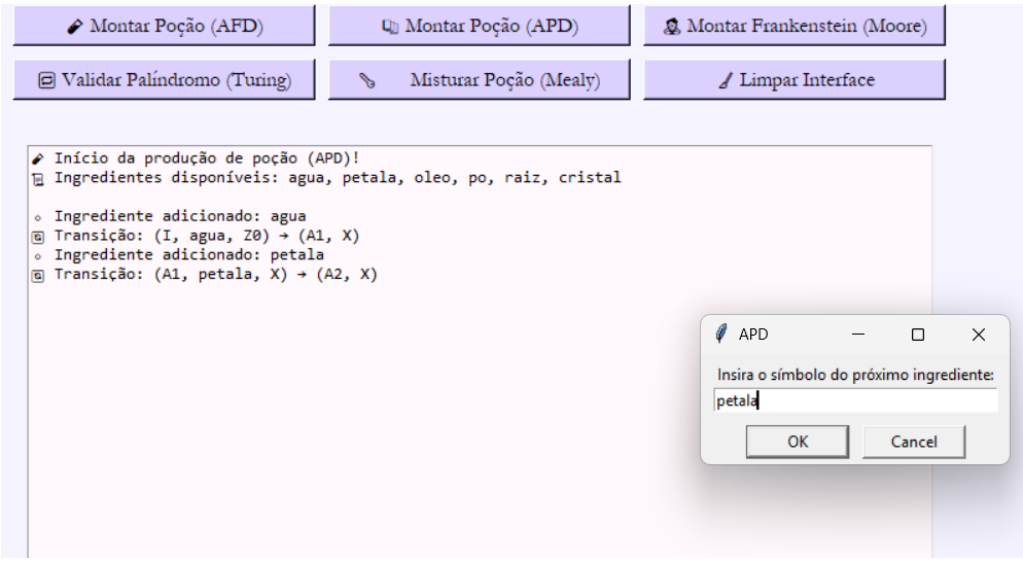

Figura 33. Adicionando ingredientes no APD.

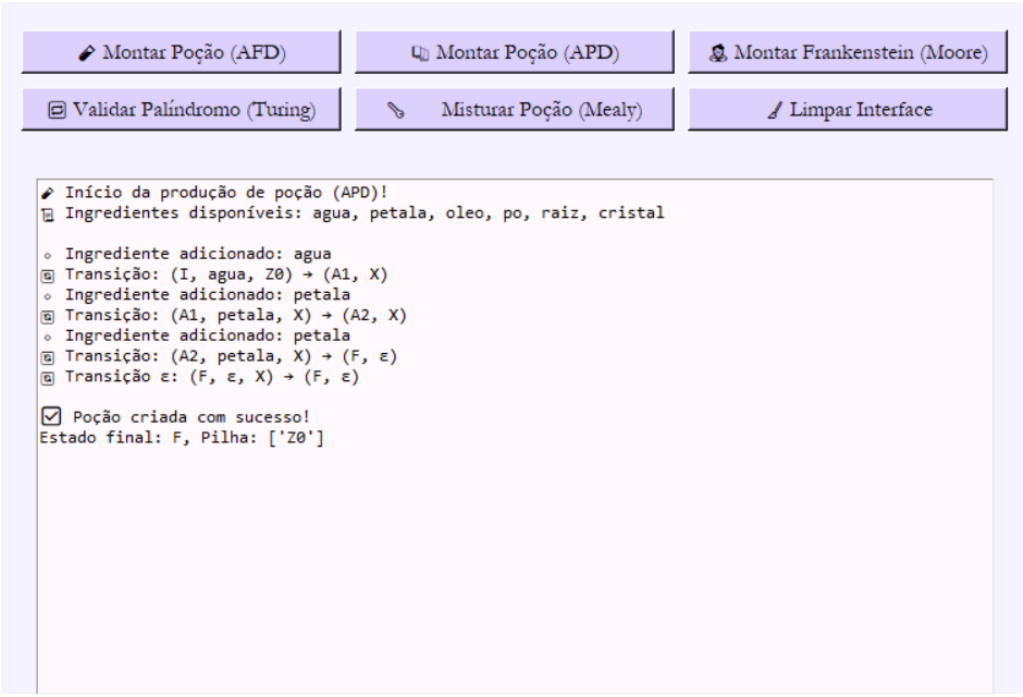

Figura 34. Depois de adicionar todos os ingredientes a poção é criada.

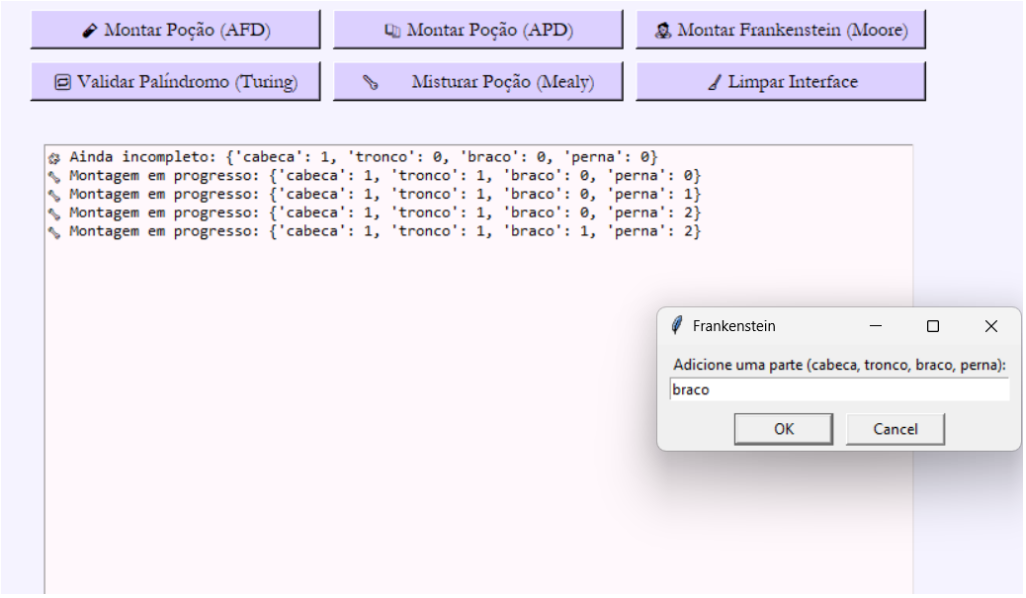

Figura 35. Adicionando as partes do corpo na máquina de Moore.

Figura 36. Frankstein criado com sucesso.

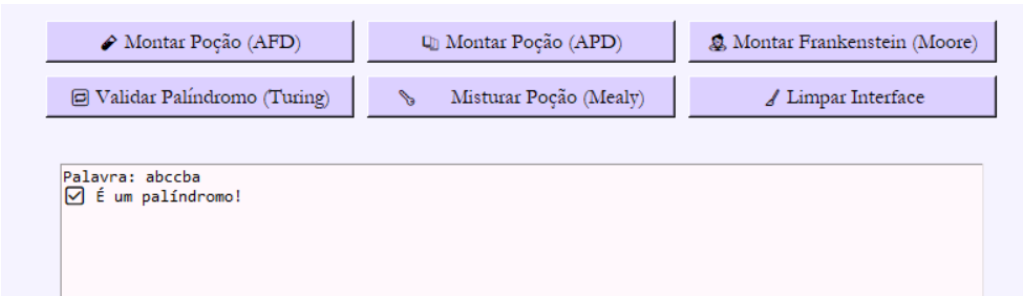

Figura 37. Validação de Palíndromo com a máquina de Turing.

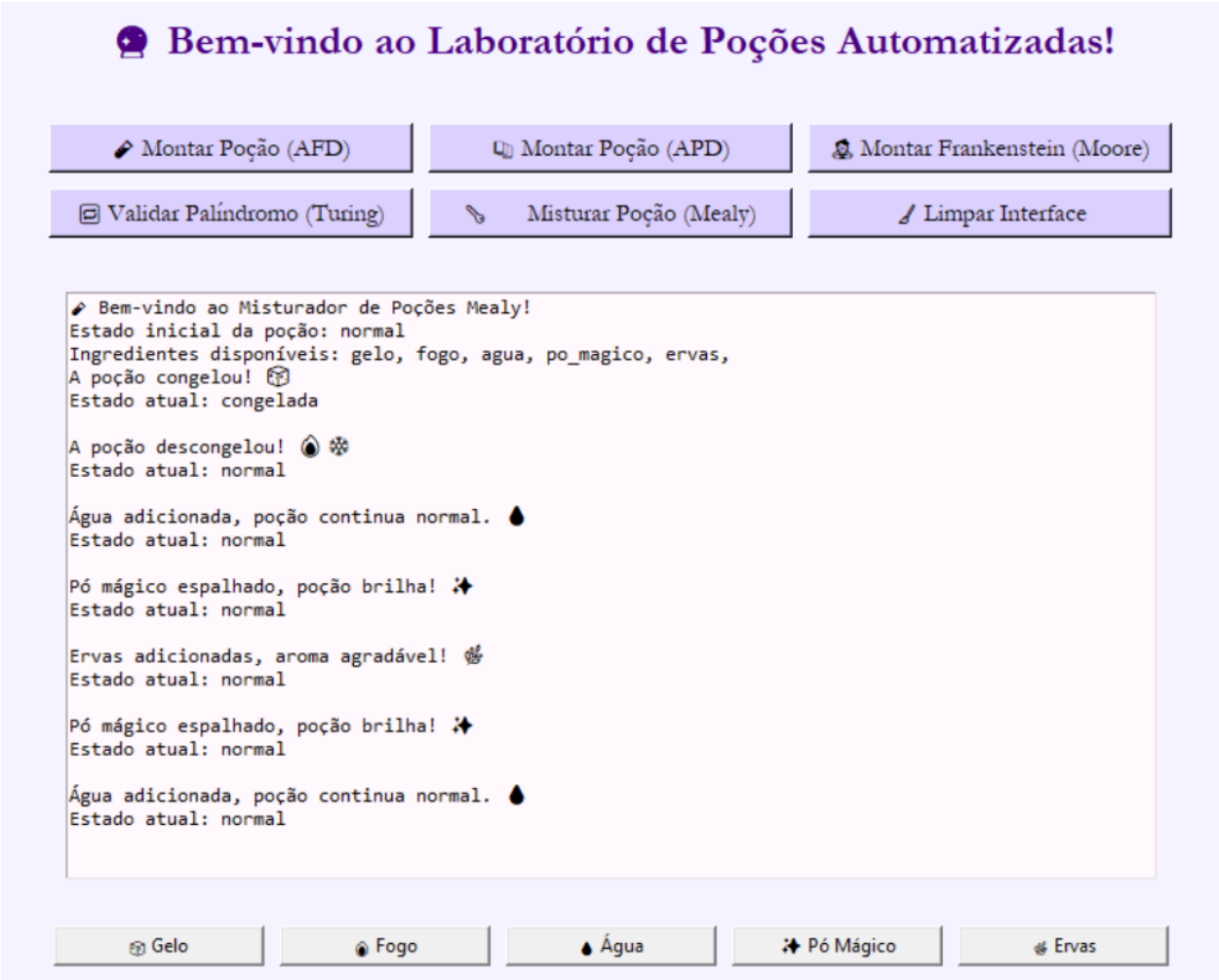

Figura 38. Transições sendo trocadas na máquina de mealy.

5. Conclusão

O desenvolvimento deste projeto proporcionou uma aplicação prática e lúdica dos principais conceitos da Teoria da Computação, permitindo a implementação de autômatos de diferentes tipos — como AFD, APD, Moore, Mealy e Máquina de Turing — de forma interativa e visual. A criação de um laboratório gráfico para montagem e validação de poções tornou o processo de manipulação dos autômatos mais didático e acessível, reforçando o entendimento das estruturas formais de reconhecimento e processamento de linguagens.

Além de consolidar o funcionamento individual de cada autômato, a integração de todos eles em uma única interface gráfica também demonstrou a importância da organização modular e da interação com o usuário. A personalização da temática mágica e o uso de elementos visuais contribuíram para tornar o projeto mais envolvente e divertido, sem comprometer a complexidade técnica por trás das implementações.

Por fim, este trabalho destacou a versatilidade dos autômatos no contexto computacional e sua relevância como base para linguagens formais, validação de padrões e simulação de algoritmos, reforçando seu papel fundamental na formação de profissionais da área de computação.

6. Referências

MENDES, Daniel. *Slides de Aula - Fundamentos da Teoria da Computação*. Universidade Federal de Viçosa - Campus Florestal, 2025.